



Carl von Ossietzky
**Universität
Oldenburg**

**„Clinical Decision Support System for
Pulmonary Embolism Detection in the
Emergency Department“**

Interface- and Workflow-Documentation

Course: Big Data Analytics und Clinical Decision
Support

Group Members:

Yeji Lee

Taki Djebbar

Ahmad Khalifeh

Rebaz El-Scheich

Document Version 1.0

07.06.2026

Inhaltsverzeichnis

Introduction	1
Purpose oft the Application	1
General Workflow	1
Project Structure	2
Main Folders	3
Interface Architecture	3
Phase 0: Patient Selection.....	4
Phase 1: Patient Overview & Data Entry.....	7
Phase 2: Wells Score & PE Risk.....	11
Phase 3: Recommendations	14
Patient History	18
<i>Data Handling</i>	20
Database Storage	23
Configuration	26
Logging.....	30
Summary.....	33

Introduction

This document describes the graphical interface and workflow of the CDSS_PE application. The application is implemented as a Clinical Decision Support System for supporting the diagnosis of pulmonary embolism. The focus of this documentation is to explain how the interface is structured, how the user moves through the different workflow steps, how patient data is handled, and which code files are responsible for the main application logic. The documentation is written from both a user and developer perspective. First, it explains the visible workflow and interaction steps. Then, it connects these steps to the relevant implementation files.

Purpose of the Application

The purpose of the application is to collect medical information in a structured way and support the clinical decision-making process through a multi-step workflow.

The application supports:

- entering patient-related information
- documenting clinical data
- guiding the user through a structured diagnostic workflow
- supporting the assessment of pulmonary embolism
- logging relevant application events
- organizing data for further processing or review

General Workflow

The application follows a multi-step workflow. The user is guided through different phases, each representing a specific part of the clinical decision support process.

The workflow starts with selecting an existing patient or registering a new patient. After that, the user enters clinical data, completes the Wells Score assessment, reviews the recommendation, and saves the completed assessment.

Simplified workflow:

Start application

1. → Patient Selection
2. → Patient Overview & Data Entry
3. → Wells Score & PE Risk
4. → Recommendations
5. → Save Assessment
6. → Return to Patient Selection→ Display or store results

At the end of the workflow, the completed assessment is saved through the database handler. The saved assessment includes patient information, clinical values, Wells Score, risk level, recommendation, physician decision, comments, and metadata. The database logic is implemented in `data/db.py`, while the patient data structure is defined in **`data/data_model.py`**.

After the assessment has been saved, the application resets the current patient data and returns to the patient selection page. This allows the user to start a new assessment or load another patient.

Project Structure

The project consists of several main folders. Each folder has a specific responsibility within the application.

CDSS_PE

- config/
- data/
- gui/
- logger/
- main.py
- requirements.txt
- README.md

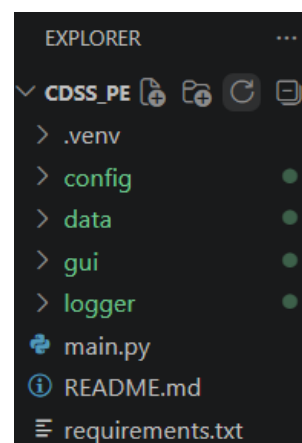


Abbildung 1 Project structure

Main Folders

Folder / File	Purpose
gui/	Contains the graphical user interface, including the main window, workflow pages, widgets, and reusable interface components.
data/	Contains the data model and database-related logic. Patient data is stored, processed, and prepared for saving here.
config/	Contains central configuration values, including texts, labels, symptoms, Wells criteria, recommendations, colours, and database/logging settings.
logger/	Contains the logging functionality. The logger records application events, navigation steps, scoring events, recommendations, physician decisions, and errors.
main.py	Starts the application and loads the graphical main window.
requirements.txt	Lists the Python dependencies required to run the project.
README.md	Contains the general project description.

This structure separates the user interface, data handling, configuration, database access, and logging logic. This makes the application easier to maintain and easier to document.

Interface Architecture

The graphical user interface is controlled by the `MainWindow` class in **gui/main_window.py**. This class represents the top-level application window and coordinates the complete multi-step workflow. The interface is built around a `QStackedWidget`. This widget allows multiple pages to be loaded into the same main window, while only one page is visible at a time. This is suitable for a step-by-step workflow because the user remains inside one main application window while moving through the different phases.

The application contains four main workflow pages:

Workflow Step	Page Class	Code Reference
Patient selection	PatientSelectPage	gui/page_patient_select.py
Data entry	DataEntryPage	gui/page_data_entry.py
Wells score assessment	WellsScorePage	gui/page_wells_score.py
Recommendations	RecommendationsPage	gui/page_recommendations.py

The MainWindow class creates these pages and adds them to the stacked widget. It also owns the shared InputHandler and DatabaseHandler objects. The InputHandler manages the currently selected patient data during the workflow, while the DatabaseHandler is responsible for loading and saving patient assessments. The navigation between pages is handled through PyQt signals. For example, when a page emits a signal such as `go_next`, the MainWindow navigates to the next workflow step.

Phase 0: Patient Selection

Purpose

The Patient Selection page is the first workflow page shown to the user. Its purpose is to either select an existing patient from the database or register a new patient before starting a pulmonary embolism assessment. This phase ensures that every assessment is linked to a specific patient record.

Interface Structure

The page is divided into two main panels:

Panel	Purpose
Registered Patients	Displays existing patients from the database and allows the user to search, select, load, or view patient history.
Register New Patient	Allows the user to create a new patient record by entering basic demographic information.

CDS PE v1.0 – Pulmonary Embolism CDSS

v1.0

1 Patient

2 Data Entry

3 Wells Score

4 Recommendations

0 Patient Selection

Patient: John Doe | MRN: 1234

REGISTERED PATIENTS

Search by name or MRN...

MRN	Name	Age	Sex	Last Assessment	PE Risk

Load Selected Patient

View Patient History

Double-click a row to load a patient, or use the buttons above.

REGISTER NEW PATIENT

MRN *

e.g. 12345678

First Name *

First name

Last Name *

Last name

Age

0 y

Sex

Other / Not specified

Triage

CTAS 1 – Resuscitation

Confirm & Start Assessment

* Required fields

Abbildung 4 Interface: Patient Data

The left panel contains a searchable patient table. The table displays patient-related summary information such as MRN, name, age, sex, last assessment, and PE risk. The user can search by name or MRN, select a row, and load the selected patient. The page also provides a button to open the patient’s assessment history.

The right panel contains the registration form for a new patient. The required fields are MRN, first name, and last name. Additional fields include age, sex, and triage.

User Process

The user can start the workflow in two ways.

Option 1: Load an existing patient

1. The user searches for a patient by MRN or name.
2. The user selects the patient from the table.
3. The user clicks **Load Selected Patient** or double-clicks the table row.
4. The selected patient is loaded from the database.
5. The application continues to the next workflow phase.

Option 2: Register a new patient

1. The user enters the MRN.
2. The user enters the first name and last name.
3. The user optionally enters age, sex, and triage information.

4. The user clicks **Confirm & Start Assessment**.
5. The application creates a new **PatientData** object and continues to the data entry

Validation

The registration form performs basic validation before a new patient is accepted. The MRN is required, and both first name and last name must be provided. If one of these required fields is missing, the application displays an input error.

If the entered MRN already exists in the database, the user is asked whether the existing patient record should be loaded instead. This prevents accidental duplicate registration of the same patient.

Patient History Acces

If an existing patient is selected, the user can open the patient history dialog. This dialog shows previous assessments for the selected patient, including assessment date, Wells Score, risk level, recommendation, physician decision, and comments. The history dialog is implemented separately in **gui/patient_history_dialog.py**.

Expected Results

After this phase, the application has an active patient record. This record is either loaded from the database or newly created from the registration form.

Once the patient is ready, the page emits a **patient_ready** signal. The **MainWindow** receives this signal, loads the patient into the shared **InputHandler**, logs the session start, and navigates to the Data Entry page.

Code Reference

Functionality	Code Reference
Patient selection page	gui/page_patient_select.py
Existing patient table	PatientSelectPage._build_left_panel()
New patient form	PatientSelectPage._build_right_panel()
Load selected patient	PatientSelectPage._on_load_patient()
Register new patient	PatientSelectPage._on_register_new()
Patient history dialog	gui/patient_history_dialog.py
Navigation to next phase	gui/main_window.py

The main content area is divided into clinical input sections. The current implementation includes sections for vital signs, laboratory and imaging values, and symptoms or risk factors. The page is implemented in **gui/page_data_entry.py** and uses reusable components from **gui/widgets.py**, such as cards, section labels, numeric input fields, and tri-state button groups.

Clinical Input Sections

Vital Signs

The vital signs section allows the user to enter key physiological values.

Field	Unit
HR	bpm
BP	mmHg
RR	/min
SpO ₂	%
Temp	°C

These fields are defined centrally in **config/config.py** as **VITAL_FIELDS**, which means the labels and units can be adjusted from the configuration file instead of being hardcoded only in the page implementation.

Labs / Imaging

The labs and imaging section is used to enter relevant diagnostic values.

Field	Unit
D-dimer	ng/mL
Troponin	ng/L
BNP	pg/mL
Creatinine	μmol/L

These fields are also defined centrally in **config/config.py** as **LAB_FIELDS**.

Symptoms and Risk Factors

The symptoms and risk factors section allows the user to document clinical findings using three possible states: YES / NO / UNKNOWN

The following symptoms and risk factors are included:

Field

- Dyspnea
- Pleuritic chest pain
- Hemoptysis
- Leg swelling
- Recent surgery / immobilization
- Prior DVT / PE
- Hormone therapy / pregnancy
- Active malignancy

The tri-state input system is implemented through the reusable **TriStateButtonGroup** component in **gui/widgets.py**. This component provides exclusive Yes, No, and Unknown buttons and emits a value change whenever the user selects an option.

User Process

The user completes this phase by reviewing the patient context and entering the available clinical information.

1. The user checks whether the correct patient is shown in the header.
2. The user selects or updates the triage category.
3. The user enters available vital signs.
4. The user enters available laboratory or imaging values.
5. The user documents symptoms and risk factors using Yes, No, or Unknown.
6. The user continues to the next workflow step.

Data Handling

Each input field is connected to the shared InputHandler. When the user changes a value, the value is staged through `stage_update()`. This means the change is collected temporarily before it is committed to the active patient record.

The PatientData class defines the fields that store the clinical input. These include vital signs, laboratory values, symptoms, and risk factors.

Simplified data flow:

User enters value

- field change is detected
- InputHandler.stage_update()
- value is stored temporarily
- data is committed when the user continues

This approach separates temporary user input from committed patient data. It helps keep the patient record consistent across the workflow.

Expected Result

After completing this phase, the application has collected the available clinical information for the selected patient. The entered data becomes available for the following Wells Score assessment. The user can then proceed to the Wells Score page, where the pulmonary embolism risk assessment is calculated based on the selected Wells criteria.

Code Reference

Functionality	Code Reference
Data entry page	gui/page_data_entry.py
Patient header	DataEntryPage._build_patient_header()
Vital signs section	DataEntryPage._build_vitals_card()
Labs / imaging section	DataEntryPage._build_labs_card()
Symptoms and risk factors	DataEntryPage._build_middle_column()
Reusable tri-state buttons	gui/widgets.py
Stored patient data fields	data/data_model.py
Central field configuration	config/config.py

Phase 2: Wells Score & PE Risk

Purpose

The Wells Score page is used to assess the patient's pre-test probability for pulmonary embolism. In this phase, the user reviews and completes the Wells criteria. Based on the selected criteria, the application calculates the Wells Score and assigns the patient to a risk category. This phase connects the clinical input from the previous step with a structured risk assessment.

Interface Structure

The Wells Score page is divided into two main areas:

Area	Purpose
Wells Criteria	Allows the user to answer each Wells criterion with Yes, No, or Unknown.
Score Breakdown	Displays the calculated score, risk level, and data completeness.

WELLS CRITERIA

	NO	YES	UNKNOWN
Clinical signs and symptoms of DVT (3.0)	Yes	No	Unknown
PE is #1 diagnosis OR equally likely (3.0)	Yes	No	Unknown
Heart rate > 100 bpm (1.5)	<input checked="" type="button" value="Yes"/>	No	Unknown
Immobilization or surgery in previous 4 weeks (1.5)	Yes	No	Unknown
Previous DVT / PE (1.5)	Yes	No	Unknown
Hemoptysis (1.0)	Yes	No	Unknown
Malignancy (active) (1.0)	Yes	No	Unknown

Total possible score: 0-12.5
Risk categories: Low ≤ 4 | Moderate 4.5-6 | High > 6

SCORE BREAKDOWN

Criteria total	1.5
Adjustment	0.0
Wells score	1.5
Risk level	LOW RISK

DATA COMPLETENESS

30%
6 of 20 data elements completed

Source: CDS PE v1.0

← Back Next →

Abbildung 6 Wells Score and Risk Management

The page is implemented in `gui/page_wells_score.py`. It uses the reusable `TriStateButtonGroup` component from `gui/widgets.py` for the Yes, No, and Unknown selections.

Wells Criteria

The application displays the seven Wells criteria for pulmonary embolism. Each criterion has a defined point value. The criteria and point values are centrally defined in `config/config.py`.

Criterion	Points
Clinical signs and symptoms of DVT	3.0
PE is the most likely diagnosis or equally likely	3.0
Heart rate > 100 bpm	1.5
Immobilization or surgery in previous 4 weeks	1.5
Previous DVT / PE	1.5
Hemoptysis	1.0
Active malignancy	1.0

Each criterion can be answered with: YES / NO / UNKNOWN. Only criteria marked as Yes contribute to the Wells Score. Criteria marked as No or Unknown do not add points to the score.

Score Calculation

The score is calculated by the **InputHandler** in `data/data_model.py`. The method **compute_wells_score()** reads the committed Wells criteria from the current **PatientData** object and adds the corresponding points for each criterion answered with Yes.

The calculated score is then assigned to one of three risk categories:

Score Range	Risk Category
≤ 4.0	LOW RISK
4.5 – 6.0	MODERATE RISK
> 6.0	HIGH RISK

These risk categories and the score explanation are also defined in `config/config.py`.

Score Display

The right side of the page displays the score result. The interface includes:

- a circular score gauge

- criteria total
- score total
- calculated risk level
- data completeness indicator

The circular score gauge provides a visual representation of the Wells Score. The color of the gauge changes depending on the assigned risk level. This makes the result easier to interpret visually.

Data Completeness

The page also displays how much of the relevant assessment data has been completed. The completeness value is based on key fields such as vital signs, symptoms, and Wells criteria.

The data completeness calculation is handled by the InputHandler through the `data_completeness()` method in `data/data_model.py`. This method counts how many relevant fields are filled and compares them to the total number of expected fields.

User Process

The user completes this phase by reviewing all Wells criteria and selecting the appropriate answer for each one.

1. The user reviews each Wells criterion.
2. The user selects Yes, No, or Unknown for each criterion.
3. The application updates the score based on the selected answers.
4. The calculated risk level is displayed.
5. The user checks the score breakdown and data completeness.
6. The user continues to the recommendations page.

Expected Result

After this phase, the application has calculated the patient's Wells Score and assigned a pulmonary embolism risk category. The calculated result is stored in the current patient data object and is used in the next phase to determine which recommendation should be displayed.

Wells criteria input

- Score calculation
- Risk category assignment
- Recommendation selection

Code Reference

Functionality	Code Reference
Wells Score page	gui/page_wells_score.py
Wells criteria display	WellsScorePage._build_criteria_card()
Score breakdown display	WellsScorePage._build_breakdown_card()
Circular score gauge	ScoreGauge in gui/page_wells_score.py
Wells criteria configuration	config/config.py
Score calculation	InputHandler.compute_wells_score() in data/data_model.py
Data completeness calculation	InputHandler.data_completeness() in data/data_model.py
Tri-state input component	TriStateButtonGroup in gui/widgets.py

Phase 3: Recommendations

Purpose

The Recommendations page is the final workflow page before the assessment is saved. Its purpose is to display a clinical recommendation based on the calculated Wells Score risk category. This phase supports the physician or user in reviewing the suggested next step, understanding the rationale behind the recommendation, and documenting whether the recommendation is accepted or overridden.

Interface Structure

The Recommendations page is divided into two main panels:

Panel	Purpose
Recommendation Panel	Displays the recommended next step, recommendation strength, rationale, and suggested actions.
Plan Decision Panel	Allows the physician to accept the recommendation, override it, select a reason for override, and add clinical comments.

CDS PE v1.0 – Pulmonary Embolism CDSSv1.0

1 Patient

2 Data Entry

3 Wells Score

4 Recommendations

3 Recommendations

Patient: John Doe | MRN: 1234

RECOMMENDED NEXT STEP

Order D-dimer (PERC rule or quantitative)

MODERATE RECOMMENDATION

RATIONALE

Patient is at low pre-test probability for PE (Wells score ≤ 4). Consider applying the PERC rule. If PERC negative, PE can be excluded without further testing. If PERC positive or not applicable, order D-dimer; proceed to CTPA only if D-dimer is elevated.

SUGGESTED ACTIONS

Apply PERC rule (if all 8 criteria negative → discharge, no further workup)

If PERC positive → Order D-dimer (quantitative)

If D-dimer negative → PE excluded

If D-dimer positive → Order CTPA

Consider alternative diagnoses for symptoms

PLAN DECISION

✓ Accept Recommendation

⚙ Override / Choose Alternative

Physician Notes / Clinical Context

Enter any additional clinical notes, override justification, or context for this assessment...

All times local

Source: CDS PE v1.0

← Back

Save & Continue to Orders →

Abbildung 7 Recommendations Page

The page is implemented in **gui/page_recommendations.py**. It receives the current patient data through the shared **InputHandler** and displays the recommendation that corresponds to the patient's Wells risk category.

Recommendation Logic

The recommendation is selected based on the calculated Wells risk level. The available risk levels are:

LOW RISK
MODERATE RISK
HIGH RISK

The recommendation content is defined centrally in **config/config.py** under **RECOMMENDATIONS**. For each risk category, the configuration contains:

Element	Description
Title	Main recommended next step.
Strength	Recommendation strength, such as strong or moderate recommendation.
Rationale	Explanation for why the recommendation is shown.
Suggested actions	List of follow-up actions related to the risk category.

This means that recommendation content can be adjusted in the configuration file without changing the page layout or GUI logic.

Recommendation Display

When the page is opened, the **populate() method** loads the current Wells risk from the patient data object. It then selects the matching recommendation from the configuration file and updates the page content.

The page displays:

- recommended next step
- recommendation strength
- clinical rationale
- suggested actions
- physician decision controls
- physician comments field

The suggested actions are displayed as selectable items, allowing the user to review which follow-up steps are relevant for the assessment.

Physician Decision

The user has two main options:

- Accept Recommendation
- Override / Choose Alternative

If the recommendation is accepted, the decision is stored as part of the current patient assessment. If the recommendation is overridden, the user can select an override reason from a predefined list. The available override reasons are defined in **config/config.py** and include options such as patient contraindication, alternative clinical judgment, resource unavailability, or other reasons. The page also provides a text area for physician notes or additional clinical context. These comments are stored with the assessment and later appear in the patient history.

User Process

The user completes this phase by reviewing the displayed recommendation and documenting the final decision.

1. The user reviews the calculated recommendation.
2. The user reads the rationale and suggested actions.
3. The user either accepts the recommendation or chooses to override it.
4. If overriding, the user selects an override reason.
5. The user optionally enters physician comments.

6. The user saves and completes the assessment.

Expected Result

After this phase, the assessment contains:

- calculated Wells Score
- Wells risk category
- selected recommendation
- recommendation strength
- physician decision
- override reason, if applicable
- physician comments, if entered

When the user saves the assessment, the **assessment_done signal** is emitted. The **MainWindow** receives this signal and saves the completed assessment through the **DatabaseHandler**.

Simplified flow:

Wells risk category

- Recommendation selection
- Physician decision
- Save assessment
- Return to patient selection

Code Reference

Functionality	Code Reference
Recommendations page	gui/page_recommendations.py
Recommendation content	RECOMMENDATIONS in config/config.py
Current patient data	InputHandler in data/data_model.py
Save assessment trigger	assessment_done signal
Final save operation	DatabaseHandler.save_assessment() in data/db.py
Logging recommendation and physician decision	logger/logger.py

Patient History

Purpose

The Patient History feature allows the user to review previous assessments for a selected patient. This supports continuity and makes it possible to compare current and past pulmonary embolism assessments. The history view is especially useful when a patient has already been assessed before and previous Wells Scores, risk levels, recommendations, or physician decisions need to be reviewed.

Access from the Interface

The Patient History function is accessed from the Patient Selection page. After selecting an existing patient from the registered patient table, the user can open the history view by clicking the **View Patient History button**. The button is only enabled when a patient row is selected. This ensures that the history dialog always opens for a specific patient.

Interface Structure

The patient history is displayed in a separate dialog window. This dialog is implemented in **gui/patient_history_dialog.py**. The dialog contains a table with previous assessments for the selected patient. The table includes the following columns:

Column	Description
Date / Time	Timestamp of the assessment
Wells Score	Calculated Wells Score
Risk Level	Assigned PE risk category
Key Data Summary	Summary of relevant clinical values
Recommendation	Recommendation shown during the assessment
Decision	Physician decision
Physician Comments	Additional comments entered by the physician

The risk level is visually highlighted in the table. High, moderate, and low risk values use different colors to improve readability and make the risk category easier to identify.

User Process

The user follows these steps to review a patient's history:

1. The user opens the Patient Selection page.
2. The user selects an existing patient from the table.
3. The user clicks **View Patient History**.
4. A separate history dialog opens.
5. The user reviews previous assessments.
6. The user can close the dialog after reviewing the information.

Export Function

The Patient History dialog includes an export function. The user can export the displayed history to an Excel file by clicking **Export to Excel**. The exported file contains the assessment history in a structured table format. This allows the assessment history to be reviewed outside the application or shared as part of project documentation or clinical review workflows. The export functionality uses **openpyxl**, which is also listed as a project dependency.

Data Source

The history data is loaded from the local SQLite database through the **DatabaseHandler**. The **method assessment_history_full()** is used to retrieve all stored assessments for the selected MRN. The history records are ordered by assessment timestamp, with the newest assessments shown first.

Simplified flow:

Selected patient

- MRN passed to PatientHistoryDialog
- DatabaseHandler loads assessment history
- History table is populated
- Optional export to Excel

Expected Result

After opening the Patient History dialog, the user can see all previous assessments stored for the selected patient. This includes the clinical result, recommendation, physician decision, and comments. If no previous assessments are available, the dialog displays a message indicating that no assessments were found for the patient.

Code Reference

Functionality	Code Reference
Open history from patient selection	gui/page_patient_select.py
Patient history dialog	gui/patient_history_dialog.py
Load full assessment history	DatabaseHandler.assessment_history_full() in data/db.py
Export to Excel	PatientHistoryDialog._on_export() and _write_xlsx()
Excel dependency	openpyxl
Database file	data/cdss_pe.db

Data Handling

Purpose

The data handling layer is responsible for storing, updating, and preparing patient-related information during the workflow. It separates the user interface from the internal patient data structure. This separation is important because the GUI pages only collect or display information, while the data layer manages how this information is stored and passed between workflow steps.

Central Data Model

The central data object is the **PatientData** class in **data/data_model.py**. This class stores all information related to one patient assessment.

The stored information includes:

Category	Examples
Demographics	MRN, first name, last name, age, sex, triage
Vital signs	HR, BP, RR, SpO ₂ , temperature
Labs / imaging	D-dimer, troponin, BNP, creatinine, imaging
Symptoms and risk factors	dyspnea, pleuritic pain, hemoptysis, leg swelling, recent surgery, malignancy
Wells criteria	DVT signs, PE likely, HR over 100, immobilization, previous DVT/PE, hemoptysis, malignancy
Computed results	Wells Score, Wells risk

Physician decision	recommendation title, recommendation strength, decision, override reason, comments
Metadata	assessment timestamp, last updated timestamp

The **PatientData class** also provides helper functionality, such as returning the patient's full name, converting the object into a dictionary, recreating an object from a dictionary, and identifying missing critical fields.

InputHandler

The current patient record is managed by the **InputHandler class**, also located in **data/data_model.py**. The **InputHandler** manages a committed patient record and a temporary staged record. This means that user input can first be collected temporarily before it is committed to the actual patient object. This mechanism is useful in a multi-step workflow because the user may enter or change data across several pages before the final assessment is saved.

Simplified structure: InputHandler

- committed PatientData
- staged changes

Staging and Committing Data

When the user changes a field in the interface, the value is passed to the **InputHandler** through **stage_update()**. The staged data is not immediately saved to the database. Instead, it is temporarily stored and later committed to the active **PatientData** object. When **commit()** is called, staged values are merged into the committed patient data and the **last_updated** timestamp is refreshed.

Simplified data flow: User input

- stage_update(field, value)
- temporary staged value
- commit()
- updated PatientData object

Use Across the Workflow

The same **InputHandler** object is shared across the workflow pages. It is created in **MainWindow** and passed to the pages that need access to patient data. This allows data entered on one page to remain available on later pages. For example, patient data entered during the Data Entry phase can be used during the Wells Score assessment and later during the recommendation phase.

The shared **InputHandler** is used by:

Page	Purpose
DataEntryPage	Stores clinical input such as vital signs, labs, symptoms, and risk factors.
WellsScorePage	Stores Wells criteria and calculates Wells Score and risk level.
RecommendationsPage	Stores the recommendation, physician decision, override reason, and comments.

Wells Score Calculation

The **InputHandler** also calculates the Wells Score. The **method compute_wells_score()** reads the committed Wells criteria from the current PatientData object. For every Wells criterion marked as Yes, the corresponding point value is added to the score. The final score is then assigned to one of three risk categories:

Score Range	Risk Category
≤ 4.0	LOW RISK
4.5 – 6.0	MODERATE RISK
> 6.0	HIGH RISK

This result is used by the Recommendations page to select the appropriate recommendation.

Data Completeness

The **InputHandler** can also calculate how complete the assessment data is. The **method data_completeness()** checks key assessment fields, including vital signs, symptoms, and Wells criteria. The result is returned as:

filled fields / total fields

This value can be used by the interface to show how much relevant assessment information has already been completed.

Expected Result

After the data handling phase, the application has a structured and updated PatientData object that contains all relevant assessment information. This object can then be passed to the database layer and saved as a completed assessment.

Code Reference

Functionality	Code Reference
Patient data object	PatientData in data/data_model.py
Current patient workflow state	InputHandler in data/data_model.py
Temporary input storage	InputHandler.stage_update()
Committing user input	InputHandler.commit()
Wells Score calculation	InputHandler.compute_wells_score()
Data completeness check	InputHandler.data_completeness()
Missing critical fields	PatientData.missing_critical_fields()

Database Storage

Purpose

The database layer is responsible for storing completed patient assessments and loading previously saved patient information. It separates permanent data storage from the graphical interface and the temporary workflow state. This allows the application to keep assessment records after the workflow has been completed and makes it possible to review previous assessments through the patient selection page and patient history dialog.

Database Handler

Database operations are handled by the **DatabaseHandler class** in **data/db.py**.

The DatabaseHandler is responsible for:

- initializing the database
- creating the required database table if it does not exist
- saving completed assessments
- listing registered patients
- loading a patient by MRN
- checking whether a patient already exists
- loading previous assessments for the patient history view

The database is implemented as a local SQLite database. The database file name is defined centrally in config/config.py as:

```
236
237   ### Database ###
238   DB_FILE_NAME = "cdss_pe.db"
239
```

Abbildung 8 Database implementation Code

Database Structure

The application uses a patients table to store assessment data. Each saved assessment contains summary fields as well as the complete patient data in JSON format.

Important stored values include:

Stored Value	Description
MRN	Medical record number used to identify the patient
First name / last name	Patient name
Age / sex / triage	Basic demographic and triage information
Wells Score	Calculated Wells Score
Wells risk	Risk category assigned from the Wells Score
Recommendation title	Recommendation shown to the user
Physician decision	Accepted or overridden recommendation
Override reason	Reason selected if the recommendation was overridden
Physician comments	Additional notes entered by the physician
Assessment timestamp	Time of the completed assessment
Last updated	Time of last update
Full data JSON	Complete serialized PatientData object

The combination of MRN and assessment_timestamp is used as a unique key. This allows several assessments to be stored for the same patient over time while keeping each assessment separate.

Saving an Assessment

At the end of the workflow, the completed assessment is saved by calling:

```
@pyqtSlot()
def _on_assessment_done(self) -> None:
    """Persist the completed assessment and return to the selection page."""
    d = self._handler.current

    # Stamp the assessment timestamp if not yet set
    if not d.assessment_timestamp:
        self._handler.stage_update(
            "assessment_timestamp",
            datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
        )
        self._handler.commit()
        d = self._handler.current

    success = self._db.save_assessment(d)
```

Abbildung 8 User activating assessment_done after saving on recommendation page

Here, **d** represents the current committed PatientData object. The database handler converts the patient data into JSON and inserts or updates the corresponding database entry.

Simplified save flow: Current PatientData

- save_assessment()
- serialize full patient data to JSON
- insert or update SQLite record
- log database event

If saving is successful, the user receives a confirmation message. If saving fails, an error message is displayed and the error is recorded by the logger.

Loading Patients

The Patient Selection page uses the database handler to load existing patient records. The method **list_patients()** returns one summary row per patient MRN, using the most recent assessment for each patient. This summary information is displayed in the registered patients table on the Patient Selection page. When the user selects and loads an existing patient, the database handler retrieves the latest full patient record and reconstructs it as a **PatientData** object.

Patient History

The database handler also supports patient history retrieval. The method **assessment_history_full()** loads all previous assessments for a selected MRN. These records are shown in the Patient History dialog. This allows the application to display previous Wells

Scores, risk categories, recommendations, physician decisions, and comments for the same patient.

Expected Result

The database layer ensures that completed assessments are stored persistently and can be retrieved later. This makes the workflow reusable across multiple patients and multiple assessment sessions.

Code Reference

Functionality	Code Reference
Database handler	DatabaseHandler in data/db.py
Database file name	DB_FILE_NAME in config/config.py
Database schema	_DDL in data/db.py
Save completed assessment	DatabaseHandler.save_assessment()
List registered patients	DatabaseHandler.list_patients()
Load selected patient	DatabaseHandler.load_patient()
Check existing MRN	DatabaseHandler.patient_exists()
Load patient history	DatabaseHandler.assessment_history_full()
Patient data serialization	PatientData.to_dict() in data/data_model.py

Configuration

Purpose

The configuration layer defines the central constants used across the application. It keeps interface labels, workflow names, button texts, field definitions, Wells criteria, recommendation texts, colors, logging settings, and database settings in one place. This makes the application easier to maintain because many visible texts and clinical configuration values can be adjusted without changing the individual GUI page files.

The main configuration file is:

config/config.py

Application Metadata

The configuration file defines the application title, version, and source label. These values are displayed in the interface header and footer areas.

Configuration Value	Purpose
APP_TITLE	Defines the title shown in the application window.
APP_VERSION	Defines the displayed application version.
SOURCE_LABEL	Defines the source label shown in the interface footer.

Workflow Configuration

The workflow steps are defined centrally in **config/config.py**. The application uses these values to display page titles and the step indicator at the top of the interface.

The workflow labels are:

Patient → Data Entry → Wells Score → Recommendations

The corresponding step titles are:

Step	Title
0	Patient Selection
1	Patient Overview & Data Entry
2	Wells Score & PE Risk
3	Recommendations

These configuration values are used by **gui/main_window.py** when updating the current page title and step indicator.

Interface Labels and Buttons

Button labels are also defined in the configuration file. This includes navigation buttons and decision buttons such as:

Button	Purpose
Back	Navigate to the previous workflow step.
Next	Continue to the next workflow step.
Proceed to Recommendations	Continue from Wells Score to Recommendations.
Save & Continue	Save the completed assessment.
Accept Recommendation	Accept the displayed recommendation.
Override / Choose Alternative	Override the displayed recommendation.
Load Selected Patient	Load an existing patient.

Confirm & Start Assessment	Register or confirm a patient and start the workflow.
----------------------------	---

Centralizing button text helps keep the interface consistent across all pages.

Symptoms and Risk Factors

The symptoms and risk factors are also defined centrally:

- Dyspnea
- Pleuritic chest pain
- Hemoptysis
- Leg swelling
- Recent surgery / immobilization
- Prior DVT / PE
- Hormone therapy / pregnancy
- Active malignancy

These definitions are used by **gui/page_data_entry.py** to build the clinical input sections dynamically.

Wells Score Configuration

The Wells criteria are defined in **config/config.py**. Each criterion consists of a data key, a display label, and a point value. The criteria are displayed in the Wells Score page, while the calculation itself is handled by the **InputHandler** in **data/data_model.py**.

Recommendation Configuration

The recommendation logic is also defined in the configuration file. Recommendations are organized by risk category:

LOW RISK
MODERATE RISK
HIGH RISK

For each risk category, the configuration defines:

Element	Description
Title	The main recommended next step.
Strength	Recommendation strength, such as strong or moderate recommendation.
Rationale	Explanation for the recommendation.
Suggested actions	Follow-up actions shown to the user.

The **RecommendationsPage** reads the current Wells risk category and loads the matching recommendation from this configuration.

Logging and Database Configuration

The configuration file also defines basic logging and database settings.

Setting	Purpose
LOG_FILE_NAME	Defines the log file name.
LOG_DATE_FORMAT	Defines the timestamp format used in log entries.
LOG_LEVEL	Defines the logging level.
DB_FILE_NAME	Defines the SQLite database file name.

This connects the configuration layer to both the logger and the database handler.

Expected Result

The configuration file acts as a central control point for many parts of the application. Interface labels, workflow steps, clinical fields, Wells criteria, recommendation texts, risk thresholds, logging settings, and database settings are all defined in one place.

This improves maintainability and makes it easier to update interface content or clinical wording without changing the core application logic.

Code Reference

Functionality	Code Reference
Main configuration file	config/config.py
Workflow step labels	STEP_LABELS
Page titles	STEP_TITLES
Vital sign fields	VITAL_FIELDS
Lab fields	LAB_FIELDS
Symptoms and risk factors	SYMPTOM_FIELDS
Wells criteria	WELLS_CRITERIA
Risk categories	RISK_LOW, RISK_MODERATE, RISK_HIGH
Recommendation content	RECOMMENDATIONS
Logger settings	LOG_FILE_NAME, LOG_DATE_FORMAT, LOG_LEVEL
Database file name	DB_FILE_NAME

Logging

Purpose

The logging layer records important application events during the workflow. It supports traceability, debugging, and later review of the application process. Logging is especially relevant in this project because the application handles a multi-step clinical workflow. By recording navigation steps, scoring events, recommendations, decisions, database events, and errors, the application makes important actions easier to reconstruct.

Logger Structure

The logging functionality is implemented in:

logger/logger.py

The logger is initialized lazily, which means it is created when it is first requested by the application. It writes log messages to both the console and a rotating log file.

The log file name is defined in **config/config.py**:

```
232     ### Logger ###
233     LOG_FILE_NAME = "cdss_pe.log"
234     LOG_DATE_FORMAT = "%Y-%m-%d %H:%M:%S"
235     LOG_LEVEL = "INFO"
236
```

Abbildung 9 Code for Logger

The generated log file is stored inside the logger folder.

Logged Events

The logger provides helper functions for different types of events.

Logged Event	Description
Session start	Records when an assessment session starts for a selected patient.
Navigation	Records movement between workflow steps.
Data saved	Records saved data fields.
Wells Score	Records the calculated Wells Score and risk level.
Recommendation	Records the generated recommendation and recommendation strength.
Physician decision	Records whether the recommendation was accepted or overridden.
Missing data	Records important missing fields.
Database event	Records database actions such as initialization, loading, and saving.
Error	Records errors with context and error type.

Logging Flow

A simplified logging process looks like this: Application event

- logging helper function
- application logger
- console output
- cdss_pe.log

For example, when the user moves from one workflow page to another, the navigation event is logged. When the Wells Score is calculated, the score and risk level are recorded. When the final assessment is saved, the database event is also logged.

Log File Handling

The logger uses a rotating file handler. This means the log file has a maximum size, and backup log files can be created automatically when the limit is reached. This prevents the log file from growing indefinitely while still preserving previous log entries.

Expected Result

The logging layer provides a structured audit trail of relevant application events. It helps developers understand how the application was used, identify errors, and verify whether key workflow steps were completed.

Code Reference

Functionality	Code Reference
Logger module	logger/logger.py
Logger initialization	get_logger()
Logger creation	_create_logger()
Session start logging	log_session_start()
Navigation logging	log_navigation()
Wells Score logging	log_wells_score()
Recommendation logging	log_recommendation()
Physician decision logging	log_physician_decision()
Missing data logging	log_missing_data()
Database event logging	log_db_event()
Error logging	log_error()

Log file configuration	LOG_FILE_NAME in config/config.py
------------------------	-----------------------------------

Summary

The CDSS_PE application is structured as a Python-based desktop application with a graphical user interface. It supports the assessment of pulmonary embolism through a guided multi-step workflow. The workflow begins with patient selection or registration, continues with structured clinical data entry, then proceeds to Wells Score assessment and risk classification. Based on the calculated risk category, the application displays a recommendation and allows the physician to accept or override it. After completion, the assessment is saved to a local SQLite database.

The application separates its main responsibilities into different folders:

Area	Responsibility
gui/	Handles the graphical interface, workflow pages, navigation, and reusable UI components.
data/	Stores the patient data model, manages the active workflow state, calculates the Wells Score, and handles database operations.
config/	Defines central configuration values such as labels, field definitions, Wells criteria, recommendation texts, colours, logging settings, and database settings.
logger/	Records relevant application events, including navigation, scoring, recommendations, physician decisions, database events, and errors.

The main workflow is coordinated by the MainWindow class in gui/main_window.py. It loads the individual workflow pages and manages navigation between them. The current patient data is managed through the shared InputHandler, while completed assessments are saved through the DatabaseHandler.

Overall, the project is organized in a modular way. The interface, data model, database logic, configuration, and logging functionality are separated from each other. This structure makes the application easier to understand, maintain, extend, and document.

The documentation provides an overview of how users interact with the application and how the main technical components support the workflow behind the interface.